

CritterOps on a Raspberry Pi — Home Database Setup

For: The Wildlife Pros shop

Goal: Run CritterOps and PostgreSQL on a Raspberry Pi at the house so the live database stays local (\$0/month for hosting).

Print this and check boxes as you go.

What you are building

```
Phones / office computers
  |
  | Shop Wi-Fi OR Tailscale (when on a job)
  ▼
Raspberry Pi at the house
├─ CritterOps app (http://PI-IP:3000)
└─ PostgreSQL (local database – system of record)
```

Important: Do **not** try to keep the app on Vercel and only put the database on the Pi. Vercel cannot reliably reach a computer on his home network. Put **both** the app and PostgreSQL on the Pi.

SQLite is optional later. Today CritterOps expects **PostgreSQL**, which is free and already matches the code.

Before you start — shopping / hardware checklist

- ☐ Raspberry Pi 5 preferred (4 GB minimum; 8 GB nicer). Pi 4 4GB+ also works.
 - ☐ Official power supply (undervoltage causes flaky Postgres).
 - ☐ MicroSD card **only for the OS** (32 GB+), class A2 if possible.
 - ☐ **USB SSD** for the database (strongly recommended — SD cards wear out).
 - ☐ Ethernet cable to the shop router (Wi-Fi works; ethernet is more stable).
 - ☐ A keyboard + HDMI screen for first boot, **or** headless setup from a laptop.
 - ☐ Laptop on the same Wi-Fi for the rest of the steps.
 - ☐ CritterOps GitHub repo access (clone URL).
 - ☐ A strong password written down somewhere safe (not “critterops”).
-

Part 1 — Flash Raspberry Pi OS

- On a laptop, install **Raspberry Pi Imager**.
- Choose **Raspberry Pi OS (64-bit)** — Lite is fine if you are comfortable with SSH; Desktop is fine if you want a screen.
- Click the gear / advanced options:
 - ☐ Enable SSH
 - ☐ Set username (example: `wildlife`)
 - ☐ Set a strong password
 - ☐ Configure Wi-Fi **or** plan to use ethernet

- ☐ Set locale / timezone to his shop timezone

4. Flash the microSD.

5. Insert the card, plug in ethernet (or Wi-Fi), power on the Pi.

6. Find the Pi on the network:

- Try `critterops.local` later after you rename it, or
- Check the router's client list for `raspberrypi`, or
- From a Mac/Linux laptop: `ping raspberrypi.local`

7. SSH in:

```
ssh wildlife@raspberrypi.local
```

(Use the username you set. If `.local` fails, use the IP from the router, e.g. `ssh wildlife@192.168.1.50`.)

Part 2 — First-time Pi hardening & updates

Run on the Pi:

```
sudo apt update && sudo apt full-upgrade -y
sudo raspi-config
```

In `raspi-config` :

- ☐ **System Options → Hostname** → set to `critterops`
- ☐ **Localisation → Timezone** → shop timezone
- ☐ Finish / reboot if asked

After reboot, SSH again:

```
ssh wildlife@critterops.local
```

Optional but smart:

```
sudo apt install -y fail2ban ufw
sudo ufw allow OpenSSH
sudo ufw allow 3000/tcp
sudo ufw enable
```

Do **not** open port `5432` to the public internet. Postgres stays on the Pi (localhost) only.

Part 3 — Attach the USB SSD (for the database)

1. Plug in the USB SSD.
2. List disks:

```
lsblk
```

3. Format and mount (example assumes the disk is `/dev/sda` — **confirm before wiping**):

```
sudo mkfs.ext4 /dev/sda1
# If the disk has no partition yet:
#   sudo parted /dev/sda --script mklabel gpt mkpart primary ext4 0% 100%
#   sudo mkfs.ext4 /dev/sda1

sudo mkdir -p /mnt/critterops-data
sudo mount /dev/sda1 /mnt/critterops-data
sudo mkdir -p /mnt/critterops-data/postgres
```

4. Make it mount on boot — get the UUID:

```
sudo blkid /dev/sda1
```

5. Add a line to `/etc/fstab` (use your real UUID):

```
UUID=YOUR-UUID-HERE /mnt/critterops-data ext4 defaults,nofail 0 2
```

6. Test:

```
sudo mount -a
df -h /mnt/critterops-data
```

Part 4 — Install Docker (easiest way to run PostgreSQL)

On the Pi:

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker "$USER"
```

Log out and SSH back in so the docker group applies.

```
docker version
```

Part 5 — Install Node.js (for CritterOps)

CritterOps needs a current Node (20+). On the Pi:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs git
node -v
npm -v
```

Part 6 — Clone CritterOps and configure the database

```
cd ~
git clone https://github.com/aipiary045-spec/The-Wildlife-Pros.git critterops
cd critterops
cp .env.example .env
```

Edit `.env` (use `nano .env`):

```
# Strong unique password – change BOTH places below to the same value
DATABASE_URL="postgresql://critterops:CHANGE_THIS_PASSWORD@127.0.0.1:5432/critterops?
schema=public"

AUTH_SECRET="paste-a-long-random-string-here"
NEXT_PUBLIC_APP_URL="http://critterops.local:3000"
```

Generate a random `AUTH_SECRET` :

```
openssl rand -hex 32
```

Paste that into `.env` .

Also create a Docker Compose override so Postgres data lives on the SSD. Create `docker-compose.override.yml` in the project folder:

```
services:
  postgres:
    environment:
      POSTGRES_USER: critterops
      POSTGRES_PASSWORD: CHANGE_THIS_PASSWORD
      POSTGRES_DB: critterops
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - /mnt/critterops-data/postgres:/var/lib/postgresql/data
```

Use the **same password** as in `DATABASE_URL` . Binding to `127.0.0.1` keeps Postgres off the LAN.

Start Postgres:

```
docker compose up -d
docker compose ps
```

You should see `critterops-postgres` healthy.

Part 7 — Load the CritterOps schema (and optional demo data)

Still in `~/critterops` :

```
npm install
npx prisma generate
npm run db:push
```

Optional demo seed (fake clients/jobs — skip if you want a clean empty shop DB):

```
npm run db:seed
```

Demo logins after seed (change these later on `/team`):

Role	Email	Password
Admin	admin@thewildlifepros.com	demo
Dispatch	dispatch@thewildlifepros.com	demo
Tech	tech@thewildlifepros.com	demo

Part 8 — Build and run the app on the Pi

```
npm run build
npm run start
```

From a phone or laptop on the same Wi-Fi, open:

```
http://critterops.local:3000
```

If `.local` does not resolve, use the Pi's LAN IP:

```
http://192.168.x.x:3000
```

Find the IP with:

```
hostname -I
```

Keep the app running after logout / reboot

Install a systemd service. Create `/etc/systemd/system/critterops.service` (use your real username/paths):

```
[Unit]
Description=CritterOps
After=network.target docker.service
Requires=docker.service

[Service]
Type=simple
User=wildlife
```

```
WorkingDirectory=/home/wildlife/critterops
EnvironmentFile=/home/wildlife/critterops/.env
ExecStart=/usr/bin/npm run start
Restart=on-failure
RestartSec=5
```

[Install]

```
WantedBy=multi-user.target
```

Then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now critterops
sudo systemctl status critterops
```

Also make sure Docker starts on boot (usually already enabled):

```
sudo systemctl enable docker
```

Part 9 — Point “everything” at the home server (this is the whole point)

There is no separate cloud database to “point at.” Once Parts 6–8 work:

- ☐ Shop computers bookmark `http://critterops.local:3000` (or the LAN IP).
- ☐ Phones on shop Wi-Fi use the same URL.
- ☐ Stop using the old Vercel app as the live system (or leave it as a read-only demo only).
- ☐ New accounts, jobs, traps, and timesheets live in Postgres **on this Pi**.

If you ever move the app to another PC on the same network later, that machine's `.env` would use:

```
DATABASE_URL="postgresql://critterops:PASSWORD@critterops.local:5432/critterops?
schema=public"
```

...but only after you intentionally open Postgres on the LAN (not required for the all-on-Pi setup above). Prefer keeping Postgres bound to `127.0.0.1` when the app runs on the same Pi.

Part 10 — Reach the shop Pi from job sites (Tailscale)

Home Wi-Fi only works at the house. For trucks / field phones:

1. Create a free Tailscale account.
2. On the Pi:

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

3. Install Tailscale on each phone and the office laptop; sign into the **same** Tailscale account/tailnet.
4. On the Pi, get the Tailscale IP (looks like `100.x.y.z`):

```
tailscale ip -4
```

5. Update `.env` :

```
NEXT_PUBLIC_APP_URL="http://100.x.y.z:3000"
```

(Or use Tailscale MagicDNS name if enabled, e.g. `http://critterops:3000` .)

6. Restart the app:

```
sudo systemctl restart critterops
```

7. On a phone with Tailscale connected, open that URL and log in.

No router port-forwarding. No exposing the shop to the open internet.

Part 11 — Nightly backups (do not skip)

CritterOps includes `scripts/backup-postgres.sh` .

On the Pi, test once:

```
cd ~/critterops
DATABASE_URL="postgresql://critterops:CHANGE_THIS_PASSWORD@127.0.0.1:5432/critterops"
./scripts/backup-postgres.sh
ls -lh backups/
```

Install Postgres client tools if `pg_dump` is missing:

```
sudo apt install -y postgresql-client
```

Add a nightly cron job:

```
crontab -e
```

Add (runs 2:15 AM local):

```
15 2 * * * cd /home/wildlife/critterops &&
DATABASE_URL="postgresql://critterops:CHANGE_THIS_PASSWORD@127.0.0.1:5432/critterops"
./scripts/backup-postgres.sh >> /home/wildlife/critterops/backups/cron.log 2>&1
```

Weekly:

- ☐ Copy `~/critterops/backups/` to a USB stick **or** Google Drive.
 - ☐ Keep at least 7 days of dumps.
-

Part 12 — Smoke test before you call it done

- ☐ Pi reboots and CritterOps comes back without SSH (`sudo systemctl status critterops`).
- ☐ Postgres is healthy (`docker compose ps`).
- ☐ Login works from a shop laptop.
- ☐ Create a test client + job on the schedule.
- ☐ Phone on shop Wi-Fi can open the app.
- ☐ Phone on cellular + Tailscale can open the app.
- ☐ Backup script produces a `.sql.gz` file.
- ☐ Change demo passwords / create real users on `/team` .
- ☐ Write down: Pi hostname, LAN IP, Tailscale IP, where backups live, and who knows the Postgres password.

Troubleshooting

Problem	Fix
Cannot SSH	Confirm Pi is powered, ethernet/Wi-Fi connected; use router IP list
<code>critterops.local</code> fails	Use hostname <code>-I</code> IP instead; some Android phones struggle with <code>.local</code>
App loads but login/API fails	Check <code>DATABASE_URL</code> password matches Docker Compose; <code>docker compose logs postgres</code>
App unreachable off-site	Tailscale must be online on Pi and phone; confirm <code>tailscale status</code>
Disk filling up	Confirm Postgres data is on the SSD (<code>docker compose volume path</code>); prune old backups
Power brownouts	Use official PSU; consider a small UPS for the Pi

Security reminders

- Change every default password (`demo` , `critterops` , etc.).
- Do not port-forward `5432` or `3000` on the home router to the public internet.
- Prefer Tailscale for remote access.
- Keep `AUTH_SECRET` private; never commit `.env` to GitHub.
- OS updates monthly: `sudo apt update && sudo apt full-upgrade -y`

One-page summary

- Flash Pi OS → SSH in → update → hostname `critterops` .
- Mount USB SSD at `/mnt/critterops-data` .
- Install Docker + Node 22.
- Clone repo → `.env` with strong `DATABASE_URL` + `AUTH_SECRET` .
- `docker compose up -d` with data on the SSD.
- `npm install` → `npm run db:push` → `npm run build` → systemd service.
- Shop uses `http://critterops.local:3000` .
- Field uses Tailscale + the `100.x` URL.

9. Nightly `backup-postgres.sh` + weekly copy off the Pi.

That is the whole path from an unboxed Raspberry Pi to CritterOps pointed at a local database at his house.